

Skills & Custom Commands

Introduced: v1.0.x as commands (March 2025) | Overhauled: v2.1.0 (January 2026)

What It Is

Skills are reusable instruction sets that extend Claude's capabilities. You write a markdown file with optional YAML frontmatter, and Claude can invoke it automatically (based on context) or you invoke it manually with `/skill-name`. Skills replaced the simpler "custom commands" system – commands still work but skills add frontmatter config, supporting files, forked execution, and hot-reload.

Why It Matters

Instead of repeating the same instructions every session ("use prettier, run tests before committing, follow our API conventions"), you encode them once as skills. They become part of Claude's toolkit: – `/deploy` – your team's exact deployment steps – `/code-review` – your review checklist and standards – `/fix-issue 456` – template for how to approach bug fixes

Quick Start (60 Seconds)

```
# Create a personal skill
mkdir -p ~/.claude/skills/greet
cat > ~/.claude/skills/greet/SKILL.md << 'EOF'
---
description: Greet the user and ask how to help
---
```

```
Say hello warmly and ask what coding task they need help with today.
EOF
```

```
# Use it in Claude Code
claude
> /greet
```

Skill Anatomy: SKILL.md

```
---
name: fix-issue                                # Optional (folder name used if omitted)
description: Fix a GitHub issue                 # When Claude should auto-invoke this
disable-model-invocation: false                 # true = only user can invoke with /
```

```

user-invocable: true           # false = only Claude can invoke
allowed-tools: Read, Grep, Bash # Tools Claude can use without asking
model: opus                   # Force a specific model
context: fork                 # Run in isolated subagent
agent: Explore                # Which subagent type (with context: fork)
argument-hint: "[issue-number]" # Shown in autocomplete
---
```

Instructions start here

```

Fix issue #$ARGUMENTS:
1. Read the issue details
2. Find the relevant code
3. Implement the fix
4. Write tests
5. Create a commit
```

Frontmatter Reference

Field	Default	Purpose
name	folder name	Slug for /name invocation
description	—	When Claude should use this (loaded into context always)
disable-model-invocation	false	true = user-only (/deploy)
user-invocable	true	false = Claude-only (background knowledge)
allowed-tools	—	Pre-approved tools (no permission prompts)
model	session default	Force opus, sonnet, or haiku
context	—	fork = run in isolated subagent
agent	—	Subagent type: Explore, Plan, general-purpose
argument-hint	—	Shown in autocomplete after /name
hooks	—	Hooks scoped to this skill's lifecycle

Configuration: Where Skills Live

Location	Path	Scope
Personal Project	~/.claude/skills/skill-name/skill.yml .claude/skills/skill-name/skill.yml	All names/skills This project (team-shared)
Plugin	plugin/skills/skill-name/skill.yml	Where SKILL is disabled
Legacy commands	.claude/commands/name.yml	Still works (no frontmatter)

If a skill and command share the same name, **skill takes precedence**.

String Substitutions

```
# Single argument
Fix issue #${ARGUMENTS}

# Positional arguments
Migrate $0 from $1 to $2
# /migrate searchBar React Vue → $0=SearchBar, $1=React, $2=Vue

# Session ID (for logging)
Log to logs/${CLAUDE_SESSION_ID}.log
```

Dynamic Context (Shell Preprocessing)

Use `!`command`` to inject live data:

```
---
name: pr-summary
context: fork
agent: Explore
---

## Current state
- Diff: !`gh pr diff`
- Changed files: !`gh pr diff --name-only`
- Comments: !`gh pr view --comments`
```

Summarize the PR changes above.

When the skill runs, each `!`command`` executes first and output replaces the placeholder.

Real-World Workflows

Workflow 1: Team Code Review Skill

```
# .claude/skills/code-review/SKILL.md
```

```
---
name: code-review
description: Review code for bugs, performance, and security
allowed-tools: Read, Grep, Glob
---
```

Review the code. Check for:

1. **Bugs**: Logic errors, null risks, off-by-one
2. **Security**: OWASP top 10, injection, auth gaps
3. **Performance**: Unnecessary loops, N+1 queries, memory leaks

For each issue: file:line, severity (critical/warning/suggestion), fix.
/code-review src/auth/

Workflow 2: Deploy with Safety Checks

```
# .claude/skills/deploy/SKILL.md
```

```
---
name: deploy
description: Deploy application to production
disable-model-invocation: true
allowed-tools: Bash, Read
---
```

```
## Pre-flight
```

- Status: `!`git status --short``
- Branch: `!`git branch --show-current``
- Last commit: `!`git log --oneline -1``

```
## Steps
```

1. Verify git status is clean (abort if not)
2. Run ``npm test`` – abort on failure
3. Run ``npm run build``
4. Deploy: ``gcloud run deploy --source .``
5. Verify health endpoint responds 200
6. Check logs for errors (last 50 lines)

Workflow 3: Subagent-Based Research

```
# ~/.claude/skills/deep-research/SKILL.md
```

```
---
name: deep-research
```

```
description: Deep research on a topic across the codebase
context: fork
agent: Explore
allowed-tools: Read, Grep, Glob
---
```

```
Research "$ARGUMENTS" thoroughly:
1. Find all relevant files
2. Read and analyze each one
3. Map dependencies and data flow
4. Summarize findings with file:line references

/deep-research "how does authentication work in this codebase"
```

Supporting Files

Keep SKILL.md focused. Reference extra files:

```
code-review/
├── SKILL.md
├── checklist.md          # Detailed review checklist
└── examples/
    ├── good-review.md    # Example of quality review
    └── bad-review.md     # What to avoid
```

In SKILL.md: See checklist.md for the full checklist.
Claude loads supporting files only when relevant, saving context.

Invocation Control

Config	You invoke	Claude invokes	Use case
Default	/name	Automatic	Most skills
disable-model-invocation: true	never	Never	Dangerous: deploy, delete
user-invocable: false	Never	Automatic	Background knowledge

How It Interacts With Other Features

Feature	Interaction
Plugins	Plugin skills are namespaced: <code>/plugin-name:skill-name</code>
Hooks	Skills can define hooks in frontmatter (scoped to skill lifecycle)
MCP	Skills can use <code>!`command`</code> to access MCP data
Sessions	Skill invocations are part of session history (resume includes them)
Permissions	<code>permissions.allow: ["Skill(deploy)"]</code> or <code>deny: ["Skill"]</code> in settings
Subagents	<code>context: fork</code> runs skill in isolated subagent

Gotchas & Tips

1. **Description is always loaded** – keep it short (~70 chars). It consumes context in every session.
 2. **Too many skills** – Claude has a context budget (2%) for skill descriptions. If you exceed it, some skills won't trigger automatically.
 3. **Hot reload** – edit SKILL.md and it takes effect immediately. No restart needed (v2.1.0+).
 4. **Arguments with spaces** – `/fix-issue "login form bug"` passes the whole quoted string as `$ARGUMENTS`.
 5. **Forked skills don't see conversation** – `context: fork` starts fresh. Include all needed context in the skill itself or via `!`command``.
 6. **Legacy commands** – `.claude/commands/review.md` still works. No frontmatter, just markdown. `$ARGUMENTS` substitution works the same.
-

Version History

Version	Date	Change
v1.0.x	Mar 2025	Custom commands (<code>.claude/commands/</code>) introduced
v2.0.0	Sept 2025	Subagent support for commands

Version	Date	Change
v2.1.0	Jan 2026	Skills overhaul: SKILL.md, YAML frontmatter, hot-reload, forked context, custom agents
v2.1.0	Jan 2026	Hooks in skill frontmatter
v2.1.0	Jan 2026	Skills no longer stop agent on denial
v2.1.45	Feb 2026	Plugin agent skills loading by bare name